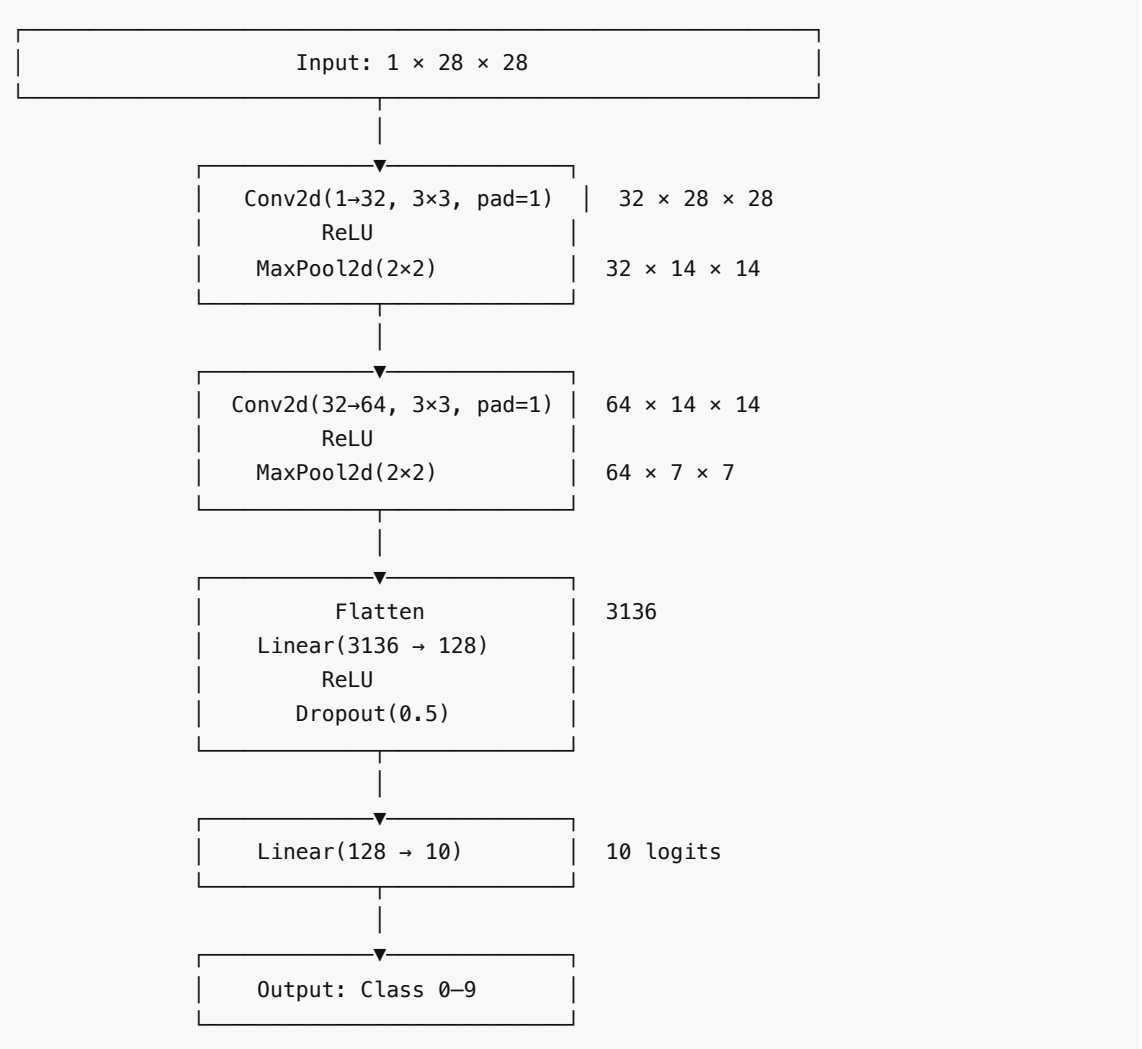
Homework 3: CNN Digit Classifier

Course: AI for Augmented and Virtual Reality (CSC 5991)
Author: Fahad Qaseem Khawar - hr4625
Dataset: MNIST — handwritten digits (0–9)
Framework: PyTorch 2.7.1
Device: Apple MPS (Metal Performance Shaders)
GitHub: github.com/fahadqaseem/cnn-digit-classifier

---

1. Network Architecture Diagram



Layer	Type	Input Shape	Output Shape	Parameters
conv1	Conv2d(1→32, 3×3) + ReLU + MaxPool	1×28×28	32×14×14	320
conv2	Conv2d(32→64, 3×3) + ReLU + MaxPool	32×14×14	64×7×7	18,496
flatten	—	64×7×7	3136	0

fc1	Linear + ReLU + Dropout(0.5)	3136	128	401,536
fc2	Linear	128	10	1,290
Total				**421,642**

Why CNN over Feedforward?

A feedforward (fully connected) network treats every pixel independently — it has no concept of spatial relationships. A CNN instead slides small **filters** (3×3 windows) across the image, learning to detect local patterns like edges and curves. These patterns are useful **wherever they appear** in the image (translation invariance). MaxPooling then compresses the feature maps, keeping only the strongest signals and reducing sensitivity to small shifts.

Result: CNNs need fewer parameters than an equivalent fully connected network, yet achieve higher accuracy on image tasks.

🔑 Key Architectural Differences vs. Assignment 2 (Feedforward)

	Assignment 2 — Feedforward	Assignment 3 — CNN
First step	Flatten image → 784 numbers	Keep image as 2D (1×28×28)
Core layer type	`nn.Linear` (fully connected)	`nn.Conv2d` (convolution)
How it sees images	Each pixel is an independent input — no spatial awareness	Slides 3×3 filters across the image — learns local patterns
Spatial structure	Destroyed immediately (flatten)	Preserved through conv layers
Translation invariance	❌ None — a "7" shifted left looks different	✅ Yes — same filter fires wherever it detects an edge
Pooling	None	MaxPool2d halves the spatial size after each conv layer
Parameters	~235K	421,642
Test accuracy	98.09%	**99.36%**

The single biggest difference: Assignment 2 flattens the 28×28 image into a 784-element vector right away and loses all spatial information. Assignment 3 keeps the image as a 2D grid and learns *where* patterns appear — that's why it outperforms the feedforward network on images.

---

2. Source Code

The full implementation is in `cnn_classifier.ipynb`.
A standalone script version is also provided: `train_cnn.py`.

Key implementation decisions

Architecture:

```
class DigitCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv1 = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2)
        )
        self.conv2 = nn.Sequential(
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.ReLU(),
            nn.MaxPool2d(2, 2)
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64 * 7 * 7, 128),
            nn.ReLU(),
            nn.Dropout(p=0.5),
            nn.Linear(128, 10)
        )

    def forward(self, x):
        x = self.conv1(x)
        x = self.conv2(x)
        return self.classifier(x)
```

Hyperparameters:

Parameter	Value	Reason
Batch size	64	Balance between memory and gradient stability
Epochs	10	Sufficient for MNIST convergence
Learning rate	0.001	Adam default, well-suited for this task
Dropout	0.5	Regularisation to prevent overfitting
Optimiser	Adam	Adaptive learning rate, faster convergence than SGD
Loss	CrossEntropyLoss	Standard for multi-class classification

---

3. Training & Testing Information

Environment:

Item	Value
Python	3.13
PyTorch	2.7.1

Device	Apple MPS (Metal Performance Shaders)
OS	macOS

Data preprocessing:

- Images converted to tensors via `transforms.ToTensor()`
- Normalised with MNIST mean (0.1307) and std (0.3081)
- Training set shuffled each epoch; test set static

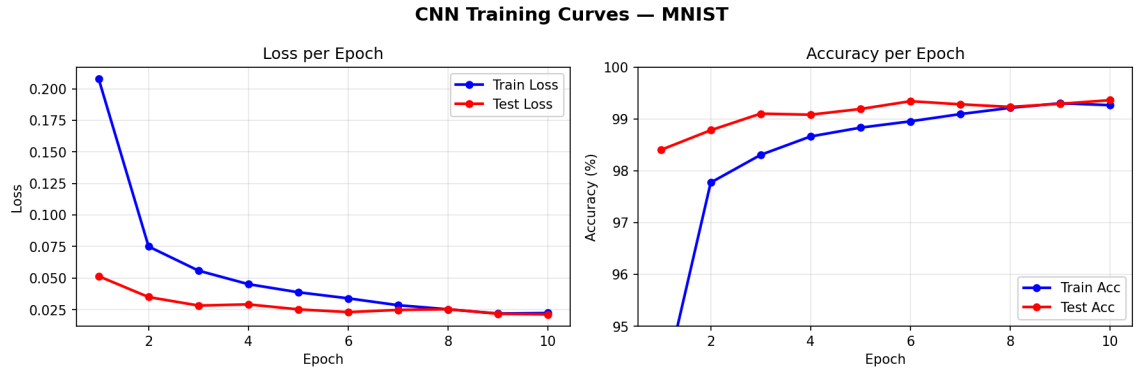
Training procedure:

1. Forward pass: image through CNN → 10 logits
2. Loss: CrossEntropyLoss between logits and true label
3. Backward pass: gradients computed via autograd
4. Weight update: Adam optimiser steps
5. Repeat for all 937 mini-batches per epoch

Training Results per Epoch

Epoch	Train Loss	Train Acc	Test Loss	Test Acc
1	0.2079	93.61%	0.0515	98.40%
2	0.0750	97.77%	0.0349	98.78%
3	0.0558	98.30%	0.0282	99.10%
4	0.0451	98.66%	0.0291	99.08%
5	0.0390	98.82%	0.0260	99.19%
6	0.0339	98.95%	0.0230	99.34%
7	0.0284	99.09%	0.0247	99.28%
8	0.0252	99.21%	0.0253	99.23%
9	0.0218	99.30%	0.0217	99.29%
10	0.0223	99.26%	0.0212	**99.36%**

Training Curves

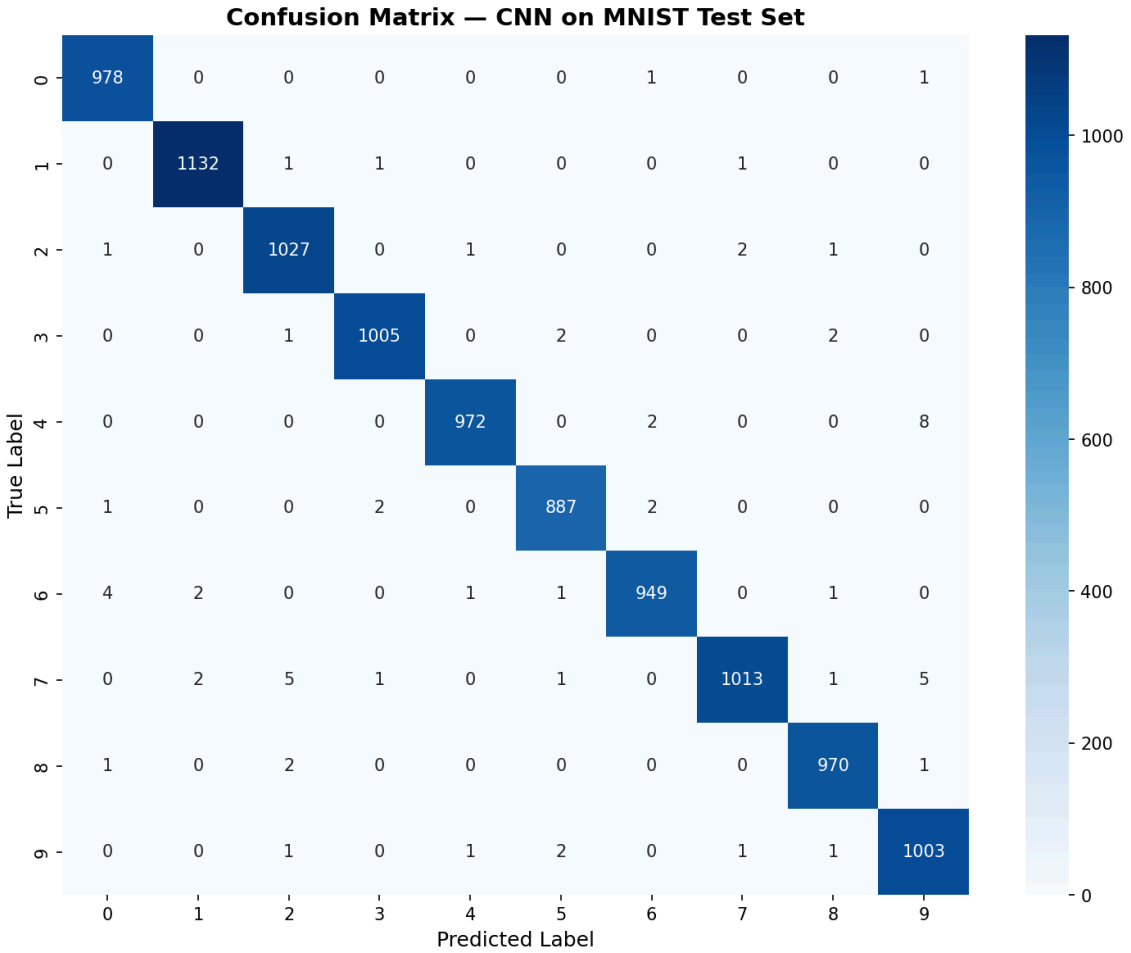


---

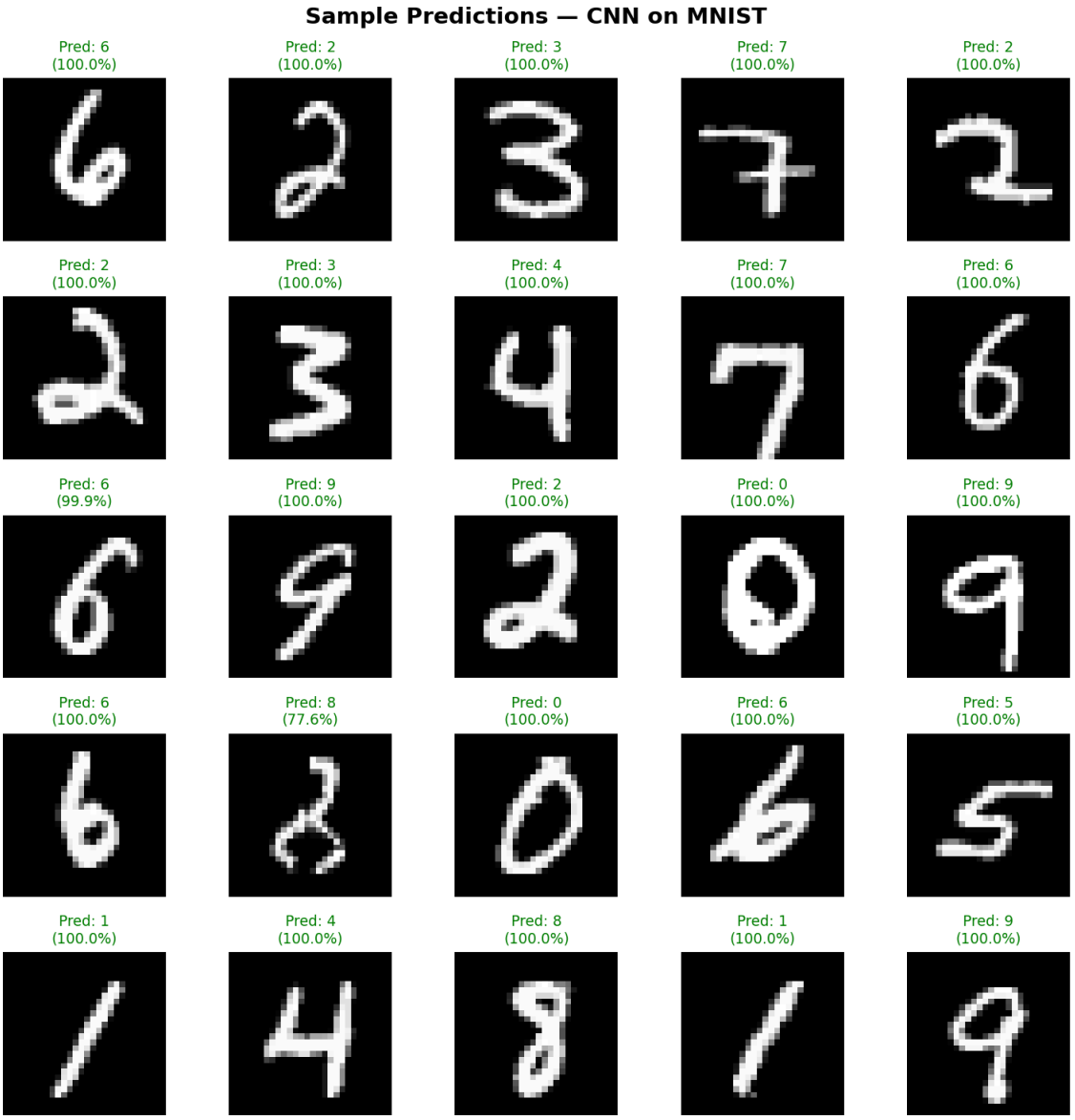
4. Test Results

Metric	Value
Test Accuracy	**99.36%**
Test Loss	0.0212
Training Time	68.4s
Total Parameters	421,642
vs. Assignment 2 (Feedforward)	98.09% → **99.36%** (+1.27%)

Confusion Matrix



Sample Predictions



Comparison: CNN vs Feedforward

Model	Architecture	Parameters	Test Accuracy
Assignment 2	FC: 784→256→128→10	~235K	98.09%
Assignment 3	2× Conv + FC head	421,642	**99.36%**

The CNN achieves higher accuracy because convolutions exploit the **spatial structure** of handwritten digits — something a flat feedforward network cannot do.

Generated with PyTorch 2.7.1 on Apple MPS